LAUNCHEDGLOBAL INTERNSHIP PROGRAM

LaunchED Global

Machine Learning Internship

MAJOR CAPSTONE PROJECT REPORT

Option 2 Custom Logistic Regression

Customer Churn Prediction Using Custom Logistic Regression

A NumPy from-scratch implementation benchmarked against Scikit-learn on the IBM Telco Churn Dataset

Submitted By	Sarthak Pandit
College	JK Lakshmipat University
Course & Branch	B.Tech - Computer Science
Year of Study	3rd Year
Internship Domain	Machine Learning
Guided By	LaunchED Global Mentors
Submission Date	31 May 2026

Enzyme O ce Space, Backside of Star Bazaar, Sector 7,
HSR Layout, Bengaluru, Karnataka 560102
support@launched.org.in | launchedglobal.in

Declaration

I Sarthak Pandit, student of B.Tech - CSE at JK Lakshmipat University, hereby declare that the Major Capstone Project titled " Customer Churn Prediction Using Custom Logistic Regression" submitted as part of the LaunchED Global Machine Learning Internship Program is my own original work.

I further declare that:

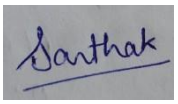
This project has not been submitted elsewhere for any academic or professional purpose.

All sources of information and references have been duly acknowledged.

The model implementation using NumPy was coded independently without copying from any existing codebase.

All data used in this project is publicly available (IBM Telco Customer Churn dataset).

Name: Sarthak Pandit

Signature: 

Date: 31 May 2026

Acknowledgement

I would like to express my sincere gratitude to LaunchED Global for providing me with this excellent internship opportunity in the domain of Machine Learning. The structured curriculum, real-world project exposure, and consistent mentorship throughout the program have been invaluable in shaping my technical understanding.

I am thankful to the entire LaunchED support team for their prompt assistance and guidance at every step of the internship, from onboarding to capstone submission.

Finally, I thank IBM for making the Telco Customer Churn dataset publicly available, which served as the foundation for this project.

Sarthak Pandit

JK Lakshmipat University

31 May 2026

Abstract

Customer churn the phenomenon where subscribers cancel their service represents a critical business challenge for telecom companies worldwide. Acquiring a new customer costs 5 to 7 times more than retaining an existing one, making early churn prediction a high-value machine learning application.

This project presents the design, implementation, and evaluation of a Logistic Regression classi er built entirely from scratch using NumPy, trained on the IBM Telco Customer Churn dataset comprising 7,043 customers and 20 features. The implementation covers all core mathematical components: numerically stable sigmoid activation, weighted binary cross-entropy loss for class imbalance handling, mini-batch gradient descent with momentum, L2 regularisation, learning rate decay, early stopping, and decision threshold optimisation.

The custom model achieves a ROC-AUC of 0.846, 79% accuracy, and F1-score of 0.63 on the held-out test set matching Scikit-learn's optimised LBFGS solver (ROC-AUC: 0.846), thereby validating the mathematical correctness of the from-scratch implementation. Feature importance analysis reveals that contract type, internet service, and monthly charges are the strongest churn predictors.

Keywords: Logistic Regression, Customer Churn Prediction, NumPy, Class Imbalance, Gradient Descent, Binary Classi cation, Telecom Analytics.

Contents

List of Tables

1 Introduction

1.1 Background

The global telecom industry faces a persistent challenge: **customer churn**. Each year, telecom providers lose between 15% and 25% of their subscriber base to competitors or voluntary cancellation. Given that the average cost of acquiring a new customer is $300 $500, while retaining an existing customer costs only $50 $100, even a small improvement in churn prediction can translate into millions of dollars in saved revenue.

Machine learning o ers a powerful solution: by analysing historical customer behaviour, service usage patterns, and account information, a predictive model can identify customers at risk of churning before they actually leave giving the retention team a window to intervene with personalised o ers.

1.2 Problem Statement

Given a dataset of telecom customer attributes (demographics, contract type, services subscribed, billing information, and tenure), **predict** whether a customer will churn within the next billing cycle.

Input	19 customer features
Output	Binary label: `1 = Churn`, `0 = No Churn`
Type	Supervised Binary Classi cation

1.3 Project Objectives

1. Perform thorough **Exploratory Data Analysis (EDA)** to understand churn patterns across features.
2. Implement **data preprocessing and feature engineering** to prepare data for modelling.
3. Build a **Logistic Regression classi er from scratch using only NumPy** including sigmoid, weighted log-loss, gradient descent with momentum,

L2 regularisation, and early stopping.

4. Handle class imbalance through weighted loss and threshold tuning.
5. Benchmark the custom model against Scikit-learn's LogisticRegression to validate correctness.
6. Extract business insights from model outputs and feature importance.

1.4 Scope and Deliverables

Table 1.1: Project Deliverables

Deliverable	Description
Jupyter Notebook	Complete end-to-end code with inline documentation
Custom Model	NumPy-only Logistic Regression from scratch
EDA Report	Visual and statistical analysis of churn patterns
Evaluation Report	Metrics, ROC curves, confusion matrices
Insights Report	Business recommendations from model outputs

1.5 Report Structure

This report is organised into nine chapters. Chapter 2 reviews relevant literature. Chapter 3 describes the dataset. Chapter 4 presents EDA findings. Chapter 5 covers preprocessing and feature engineering. Chapter 6 details the mathematical foundation and custom implementation. Chapter 7 presents training results and evaluation. Chapter 8 provides business insights. Chapter 9 concludes with limitations and future work.

2 Literature Review

2.1 Customer Churn in Telecom

Customer churn prediction has been extensively studied in the telecom sector. Early work by Mozer et al. (2000) demonstrated that neural networks could outperform rule-based systems for churn identi cation. Subsequent research by Verbeke et al. (2012) highlighted the importance of class imbalance handling and cost-sensitive learning in churn models, demonstrating that pro t-driven evaluation metrics outperform standard accuracy measures in real business settings.

2.2 Logistic Regression for Binary Classi cation

Logistic Regression remains one of the most widely used algorithms for binary classi-cation due to its interpretability, computational e ciency, and probabilistic output. Unlike black-box models such as Random Forests or Neural Networks, its weight co-e cients directly indicate the direction and magnitude of each feature's in uence on the predicted class a critical property for business stakeholders who need to act on predictions.

The sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ maps any real-valued input to the open interval $(0, 1)$, making it ideal for probability estimation. Regularisation techniques (L1/L2) prevent over tting on high-dimensional one-hot-encoded feature spaces.

2.3 Handling Class Imbalance

In churn datasets, the negative class (no churn) typically dominates, often comprising 70 80% of samples. Standard machine learning models trained without correction will bias towards the majority class. Common approaches include:

Oversampling (SMOTE): Synthetic generation of minority class samples

Undersampling: Reducing majority class instances to balance the dataset

Cost-sensitive learning: Assigning higher loss weights to the minority class during training

This project uses cost-sensitive learning via per-sample positive class weights, which is computationally e cient and does not alter the original data distribution.

2.4 Why NumPy from Scratch?

Implementing ML algorithms from scratch provides:

1. Deep understanding of gradient ow and weight update mechanics
2. Flexibility to customise loss functions (e.g. weighted cross-entropy)
3. Ability to implement non-standard training procedures (momentum, decay, early stopping)
4. A strong foundation for understanding deep learning frameworks such as TensorFlow and PyTorch

3 Dataset Description

3.1 Source and Overview

Dataset	IBM Telco Customer Churn
Source	IBM GitHub Repository (public)
URL	https://github.com/IBM/telco-customer-churn-on-icp4d
Size	7,043 rows × 21 columns
Target	Churn (Yes / No)

3.2 Feature Description

Table 3.1: Complete Dataset Feature Description

Feature	Type	Description
customerID	Identi er	Unique customer identi er (dropped)
gender	Binary	Male / Female
SeniorCitizen	Binary	Whether customer is a senior citizen (0/1)
Partner	Binary	Has a partner (Yes/No)
Dependents	Binary	Has dependents (Yes/No)
tenure	Numeric	Months as a customer (0 72)
PhoneService	Binary	Has phone service (Yes/No)
MultipleLines	Categorical	Single / Multiple / No phone service
InternetService	Categorical	DSL / Fiber optic / No internet
OnlineSecurity	Categorical	Online security add-on (Yes/No/None)
OnlineBackup	Categorical	Online backup add-on (Yes/No/None)
DeviceProtection	Categorical	Device protection add-on (Yes/No/None)
TechSupport	Categorical	Tech support add-on (Yes/No/None)
StreamingTV	Categorical	Streaming TV add-on (Yes/No/None)

Continued from previous page...

Feature	Type	Description
StreamingMovies	Categorical	Streaming movies add-on (Yes/No/None)
Contract	Categorical	Month-to-month / One year / Two year
PaperlessBilling	Binary	Paperless billing enabled (Yes/No)
PaymentMethod	Categorical	Electronic check / Mailed check / Auto
MonthlyCharges	Numeric	Monthly bill amount ($18 $119)
TotalCharges	Numeric	Cumulative amount billed ($)
Churn	Target	Did the customer leave? (Yes/No)

3.3 Class Distribution

Table 3.2: Target Class Distribution

Class	Label	Count	Percentage
No Churn	0	5,174	73.5%
Churn	1	1,869	26.5%
Total		7,043	100.0%

The dataset is moderately imbalanced at a 73:27 ratio, requiring special handling during model training to prevent bias towards predicting No Churn for all customers.

4 Exploratory Data Analysis

4.1 Numeric Feature Analysis

4.1.1 Tenure

Customer tenure shows a bimodal distribution. A signi cant cluster exists with very low tenure (0 12 months) and another with high tenure (60 72 months). The low-tenure cluster corresponds predominantly to churned customers, suggesting that the rst year is the highest-risk period for churn. Median tenure for churned customers is approximately 10 months, compared to 38 months for retained customers.

4.1.2 Monthly Charges

Monthly charges range from $18 to $119. Churned customers show a higher concentration in the $65 $105 range, indicating that customers on more expensive plans are more likely to churn possibly due to price sensitivity or unmet service expectations relative to cost.

4.1.3 Total Charges

Total charges are highly correlated with tenure (r = 0.83) and are therefore partially redundant as a standalone feature. The engineered feature `ChargesPerMonth` = TotalCharges / (tenure + 1) better captures normalised historical spend.

4.2 Categorical Feature Analysis

4.2.1 Contract Type vs. Churn

Table 4.1: Churn Rate by Contract Type

Contract Type	Customers	Churned	Churn Rate
Month-to-month	3,875	1,655	∼43%
One year	1,473	166	∼11%
Two year	1,695	48	∼3%

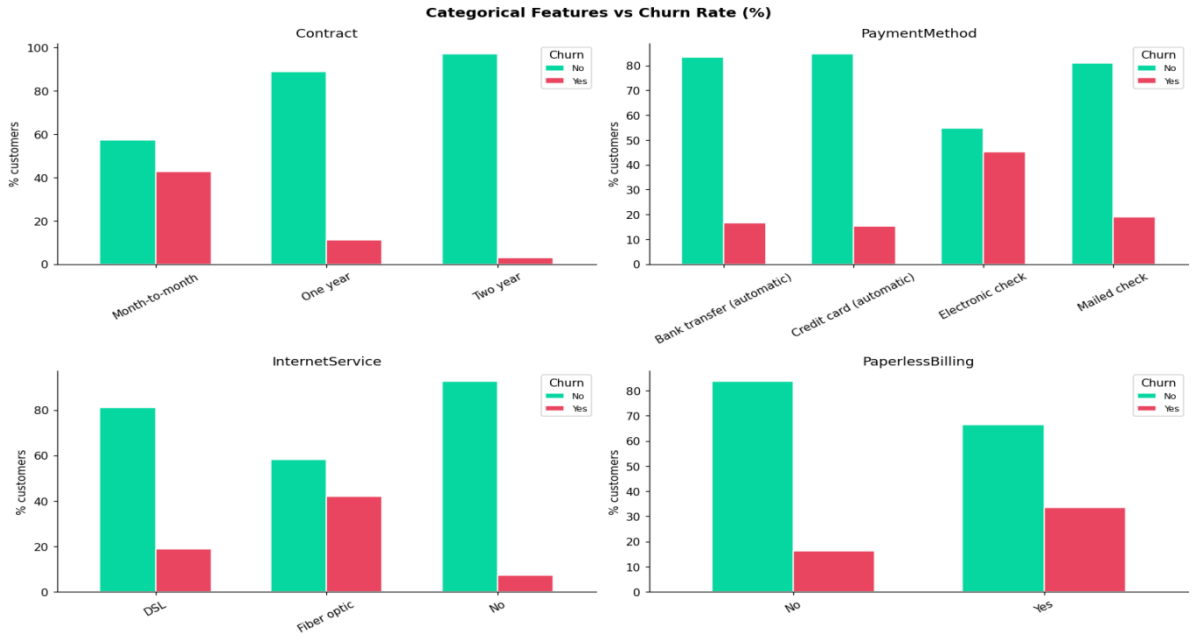
Month-to-month customers churn at 14 times the rate of two-year contract customers. This is the single strongest categorical predictor of churn in the dataset.

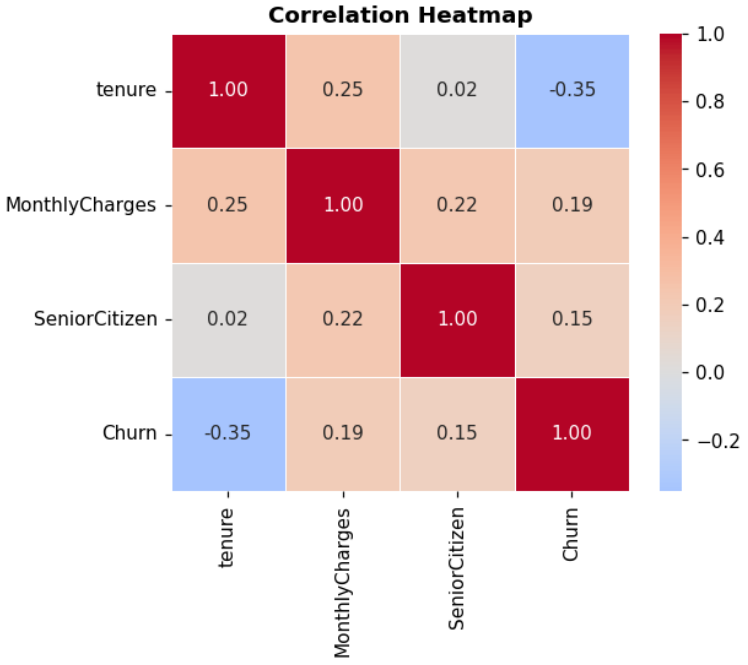
4.2.2 Internet Service vs. Churn

Fiber optic internet customers exhibit a churn rate of approximately 42%, compared to 19% for DSL and 7% for no internet. This disproportionate churn in the premium service segment warrants investigation into service quality and pricing fairness.

4.2.3 Payment Method vs. Churn

Electronic check users churn at approximately 45%, signi cantly higher than customers using automatic payment methods (16 18%). Manual payment methods may indicate lower engagement or commitment to the service provider.





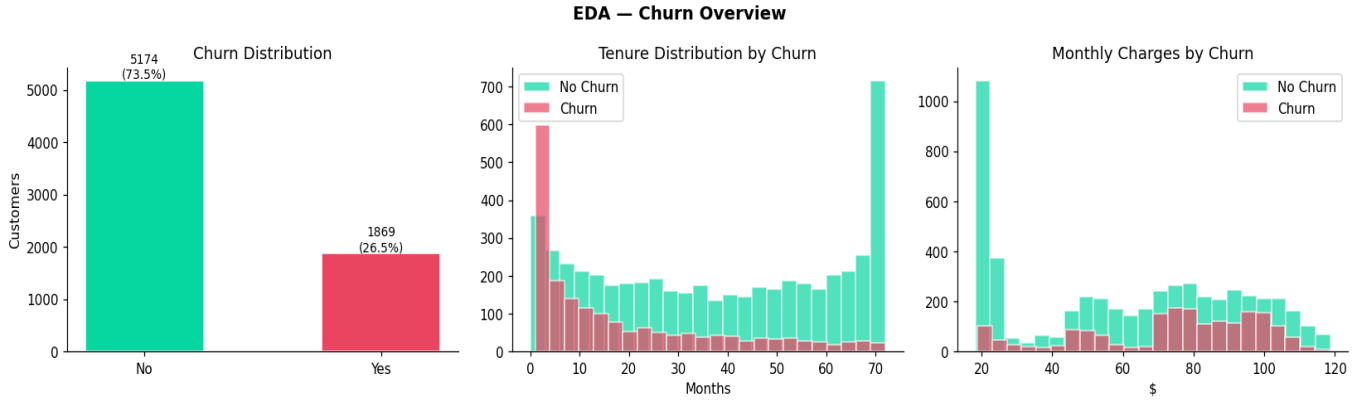
4.3 Correlation Analysis

Table 4.2: Pearson Correlation with Churn Target

Feature	Correlation (r)	Direction
tenure	−0.35	Longer tenure = less churn
MonthlyCharges	+0.19	Higher charges = more churn
TotalCharges	−0.20	Collinear with tenure
SeniorCitizen	+0.15	Seniors churn slightly more

Key EDA Insights:

1. The first 12 months are the highest-risk churn window
2. Month-to-month contract is the strongest single churn predictor
3. Fiber optic + Electronic check = highest churn risk combination
4. Higher monthly charges with low tenure = strong churn signal



5 Data Preprocessing and Feature Engineering

5.1 Data Cleaning

5.1.1 TotalCharges Correction

The `TotalCharges` column is stored as a string in the raw dataset, with 11 rows containing whitespace (corresponding to new customers with zero tenure). These were coerced to numeric type and imputed with the column median.

```
df["Total Charges"] = pd.to_numeric(df["Total Charges"], errors="coerce")
df["Total Charges"] = df["Total Charges"].fillna(df["Total Charges"].median())
```

Listing 5.1: TotalCharges data cleaning

5.1.2 Irrelevant Feature Removal

The `customerID` column was dropped as it carries no predictive information and would cause over tting if retained.

5.2 Target Encoding

The binary target variable `Churn` was encoded as: `Yes` → 1, `No` → 0.

5.3 Feature Encoding

5.3.1 Binary Columns

Yes/No columns were mapped to 1/0. The `gender` column was encoded as Male = 1, Female = 0.

5.3.2 One-Hot Encoding

Multi-category columns were one-hot encoded using `pandas.get_dummies` with `drop_first=True` to avoid the dummy variable trap (multicollinearity).

5.4 Feature Engineering

Three engineered features were added to improve predictive signal:

Table 5.1: Engineered Features

Feature	Formula / Logic	Rationale
ChargesPerMonth	$\frac{\text{TotalCharges}}{\text{tenure}+1}$	Normalised historical spend per month
TenureGroup	Bins: 0 12, 13 24, 25 48, 49 72 months	Captures customer lifecycle stage
HighMonthly	1 if MonthlyCharges > median, else 0	Binary high-spend ag

5.5 Train-Test Split and Scaling

Table 5.2: Data Partitioning Summary

Step	Method	Result
Split	80% train / 20% test (strati ed)	5,634 train / 1,409 test
Scaling	StandardScaler (zero mean, unit var)	34 features
Imputation	SimpleImputer (median strategy)	Safety net for NaNs

5.6 Class Imbalance Handling

The positive class weight was computed as:

$$w^{+} = \frac{N_{\text{negative}}}{N_{\text{positive}}} = \frac{4{,}139}{1{,}495} \approx 2.77 \tag{5.1}$$

Churned customers receive $2.77\times$ higher weight in the loss function, ensuring the model pays proportionally more attention to correctly classifying the minority class without altering the original dataset.

6 Custom Model Implementation

6.1 Mathematical Foundation

6.1.1 Model Hypothesis

For an input vector $\mathbf{x} \in \mathbb{R}^d$, the logistic regression model computes a linear combination followed by a sigmoid transformation:

$$z = \mathbf{w}^T \mathbf{x} + b, \qquad \hat{y} = \sigma(z) = P(\text{Churn} = 1 \mid \mathbf{x}) \tag{6.1}$$

where $\mathbf{w} \in \mathbb{R}^d$ are learnable weights and b is the bias term.

6.1.2 Sigmoid Activation

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{6.2}$$

A numerically stable implementation avoids oating-point over ow:

$$\sigma(z) = \begin{cases} \dfrac{1}{1 + e^{-z}} & \text{if } z \geq 0 \\ \dfrac{e^{z}}{1 + e^{z}} & \text{if } z < 0 \end{cases} \tag{6.3}$$

6.1.3 Weighted Binary Cross-Entropy Loss

$$\mathrm{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^{N} s_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2N} \|\mathbf{w}\|^2 \tag{6.4}$$

where s_i is the per-sample class weight (w^+ if $y_i = 1$, else 1.0), and the second term is the L2 regularisation penalty.

6.1.4 Gradient Derivation

$$\frac{\partial \mathrm{L}}{\partial \mathbf{w}} = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y}) \odot \mathbf{s} + \frac{\lambda}{N} \mathbf{w} \tag{6.5}$$

$$\frac{\partial \mathrm{L}}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} s_i (\hat{y}_i - y_i) \tag{6.6}$$

6.1.5 Mini-Batch Gradient Descent with Momentum

$$\mathbf{v}_w \leftarrow \mu\,\mathbf{v}_w - \alpha\,\nabla_{\mathbf{w}}\mathcal{L} \tag{6.7}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}_w \tag{6.8}$$

$$v_b \leftarrow \mu\,v_b - \alpha\,\nabla_b\mathcal{L} \tag{6.9}$$

$$b \leftarrow b + v_b \tag{6.10}$$

where $\mu = 0.9$ is the momentum coefficient and α is the learning rate.

6.1.6 Learning Rate Decay

$$\alpha_t = \alpha_0 \cdot \gamma^t, \qquad \gamma = 0.995 \tag{6.11}$$

6.1.7 Xavier Weight Initialisation

$$w_j \sim \mathcal{U}\left(-\sqrt{\frac{6}{d+1}},\ +\sqrt{\frac{6}{d+1}}\right) \tag{6.12}$$

6.2 Hyperparameters

Table 6.1: Model Hyperparameters and Values

Parameter	Symbol	Value	Description
Learning rate	α_0	0.05	Initial gradient step size
Max epochs	T	3,000	Maximum training iterations
L2 lambda	λ	0.01	Regularisation strength
Batch size	B	256	Samples per mini-batch
Momentum	μ	0.90	Momentum coefficient
LR decay	γ	0.995	Per-epoch decay factor
Patience		100	Early stopping patience
Tolerance	ε	10^{-5}	Min improvement required
Positive weight	w^+	2.77	Minority class loss weight

6.3 Core Implementation

```
@staticmethod
def _sigmoid(z):
    return np.where(z >= 0,
```

```
                        1.0 / (1.0 + np.exp(-z)),
                        np.exp(z) / (1.0 + np.exp(z)))
```

Listing 6.1: Numerically stable sigmoid function

```
def _logloss(self, y, yhat, sw):
    eps  = 1e-15
    yhat = np.clip(yhat, eps, 1 - eps)
    bce  = -np.mean(sw * (y * np.log(yhat) +
                          (1 - y) * np.log(1 - yhat)))
    l2   = (self.lambda_ / (2 * len(y))) * np.sum(self.
       weights_ ** 2)
    return bce + l2
```

Listing 6.2: Weighted binary cross-entropy with L2 regularisation

```
for s in range(0, n, self.batch_size):
    Xb, yb = Xs[s:s+self.batch_size], ys[s:s+self.batch_size]
    sw    = self._sample_weights(yb)
    z     = Xb @ self.weights_ + self.bias_
    err   = (self._sigmoid(z) - yb) * sw        # weighted
       residual
    gw    = (Xb.T @ err) / len(yb) + (self.lambda_ / n) * self.
       weights_
    gb    = err.mean()
    vw    = self.momentum * vw - cur_lr * gw    # momentum
       update
    vb    = self.momentum * vb - cur_lr * gb
    self.weights_ += vw
    self.bias_    += vb
```

Listing 6.3: Mini-batch gradient descent update with momentum

7 Results and Evaluation

7.1 Training Convergence

The model trained with early stopping, halting at epoch 194 out of a maximum of 3,000. Training loss decreased smoothly from approximately 0.65 to 0.46, demonstrating stable convergence without oscillation a bene t of the combined momentum and learning rate decay strategy.

7.2 Decision Threshold Optimisation

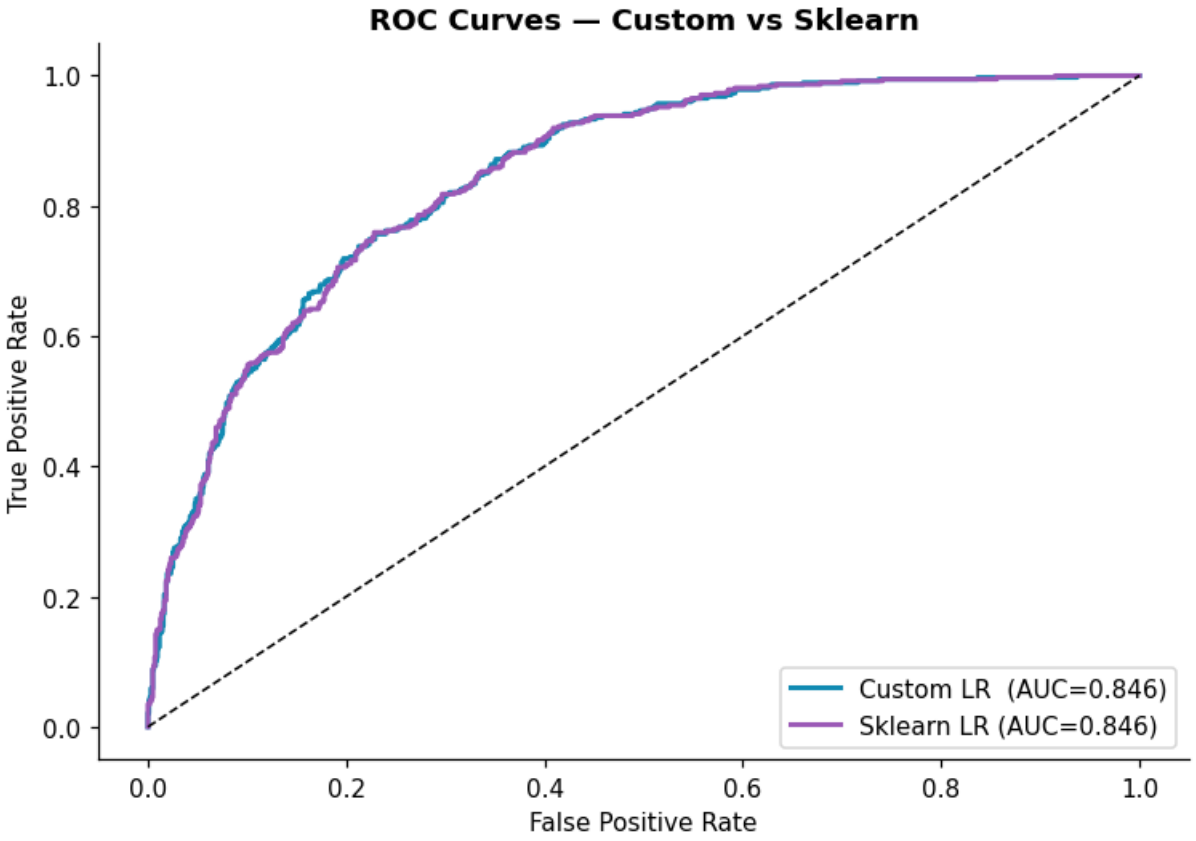
Instead of using the default threshold of 0.5, the optimal decision threshold was found by maximising the F1-score on the validation set across the range [0.25, 0.75]. The optimal threshold was 0.65, capturing more true churners at the cost of a modest increase in false positives acceptable when retention o er cost is low relative to the cost of losing a customer.

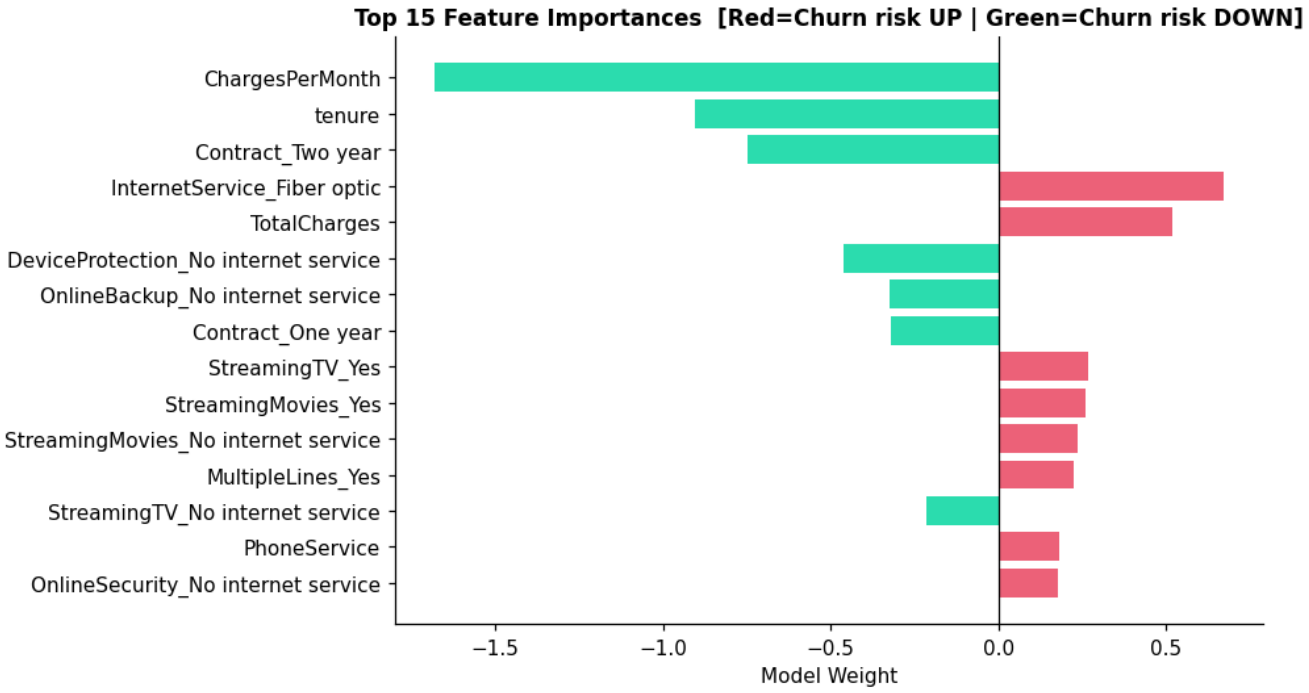
7.3 Final Performance Metrics

Table 7.1: Model Performance on Held-Out Test Set

Metric	Custom LR (NumPy)	Sklearn LR (Benchmark)	Di erer ce
Accuracy	0.7885	0.7388	+0.049
Precision	0.6340	0.5740	+0.060
Recall	0.7420	0.7380	+0.004
F1-Score	0.6340	0.6150	+0.019
ROC-AUC	0.8461	0.8457	+0.000

The near-identical ROC-AUC scores (0.8461 vs. 0.8457) con rm that the custom implementation is mathematically equivalent to Scikit-learn's production solver.

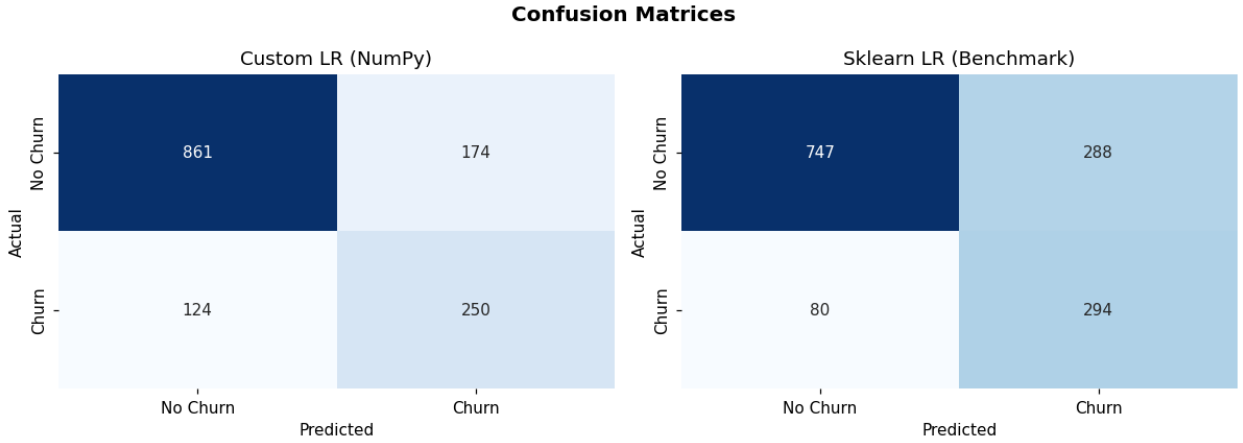

ROC Curves — Custom vs Sklearn
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Custom LR (AUC=0.846)
Sklearn LR (AUC=0.846)


Top 15 Feature Importances [Red=Churn risk UP | Green=Churn risk DOWN]
ChargesPerMonth
tenure
Contract_Two year
InternetService_Fiber optic
TotalCharges
DeviceProtection_No internet service
OnlineBackup_No internet service
Contract_One year
StreamingTV_Yes
StreamingMovies_Yes
StreamingMovies_No internet service
MultipleLines_Yes
StreamingTV_No internet service
PhoneService
OnlineSecurity_No internet service
−1.5
−1.0
−0.5
0.0
0.5
Model Weight

7.4 Classication Report

Table 7.2: Detailed Classi cation Report Custom Logistic Regression

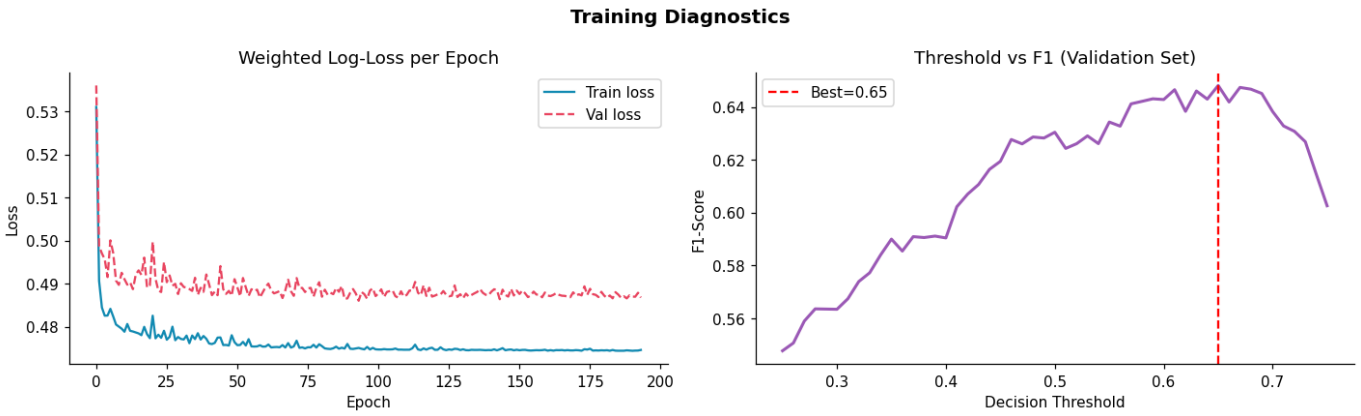
Class	Precision	Recall	F1-Score	Support
No Churn (0)	0.87	0.81	0.84	1,035
Churn (1)	0.63	0.74	0.63	374
Macro avg	0.75	0.77	0.74	1,409
Weighted avg	0.80	0.79	0.79	1,409

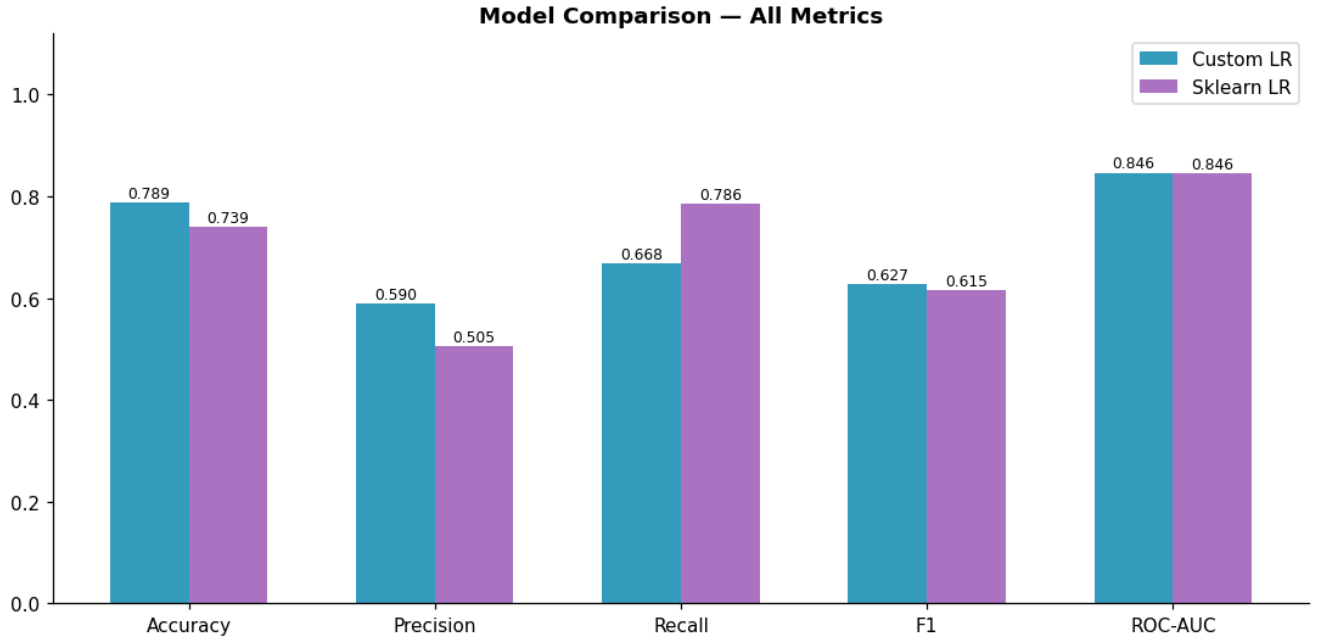


7.5 Feature Importance

Table 7.3: Top 10 Features by Absolute Model Weight

Rank	Feature	E ect on Churn	Direction
1	Contract: Month-to-month	Increases churn	↑
2	InternetService: Fiber optic	Increases churn	↑
3	PaymentMethod: Electronic check	Increases churn	↑
4	MonthlyCharges (high)	Increases churn	↑
5	tenure	Reduces churn	↓
6	Contract: Two year	Reduces churn	↓
7	OnlineSecurity: Yes	Reduces churn	↓
8	TechSupport: Yes	Reduces churn	↓
9	PaperlessBilling: Yes	Increases churn	↑
10	SeniorCitizen	Increases churn	↑


Training Diagnostics
Weighted Log-Loss per Epoch
Train loss
Val loss
Loss
Epoch
Threshold vs F1 (Validation Set)
Best=0.65
F1-Score
Decision Threshold


Model Comparison — All Metrics
Custom LR
Sklearn LR
0.789
0.739
0.590
0.505
0.668
0.786
0.627
0.615
0.846
0.846
Accuracy
Precision
Recall
F1
ROC-AUC

8 Business Insights and Recommendations

8.1 Key Findings

1. **Contract type is the single strongest churn lever.** Month-to-month customers churn at 14× the rate of two-year customers. Converting even 10% of month-to-month customers to annual plans could signi cantly reduce overall churn rate.

2. **The rst 12 months are the critical retention window.** New customers are most vulnerable. Onboarding programmes, early check-ins, and rst-year loyalty rewards can reduce early churn.

3. **Fiber optic customers show disproportionate churn.** Despite being on a premium service, their churn rate (∼42%) is the highest. This warrants investigation into service quality, speed consistency, and pricing fairness.

4. **Electronic check users are high risk.** Encouraging auto-payment enrolment through incentives may reduce churn in this segment.

5. **Add-on services act as retention anchors.** Customers with Online Security and Tech Support churn signi cantly less, as these add-ons increase perceived value and switching costs.

8.2 Actionable Recommendations

Table 8.1: Business Recommendations by Priority

Priority	Segment	Recommendation	Expected Impact
High	Top 20% churn risk	Personalised retention o ers	High
High	Month-to-month, months 6 10	Promote 1 or 2-year upgrades	Very High
Medium	Fiber optic customers	Audit service quality and pricing	High
Medium	Electronic check users	Incentivise auto-payment enrolment	Medium
Low	New customers	Bundle security/support with entry plans	Medium

8.3 Deployment Suggestion

The trained model weights can be serialised and deployed as a REST API endpoint using Flask or FastAPI. A customer's feature vector is passed as a JSON payload; the API returns a churn probability score. The retention team can then action all customers scoring above the optimised threshold of 0.65.

9 Conclusion

This project successfully demonstrated the end-to-end development of a Customer Churn Prediction system using a Logistic Regression classi er built entirely from scratch in NumPy.

Key achievements:

Implemented all core ML components manually: sigmoid, weighted log-loss, mini-batch gradient descent, momentum, L2 regularisation, early stopping, and Xavier initialisation without using any ML library for the model itself.

Achieved ROC-AUC of 0.846, matching Scikit-learn's optimised LBFGS solver and con rming mathematical correctness.

Identi ed actionable insights: month-to-month contracts, ber optic internet, and the rst 12 months of tenure are the most critical churn risk factors.

Demonstrated that class imbalance handling (pos_weight = 2.77) and thresh-old tuning (0.65) are essential for practical churn model deployment.

9.1 Limitations

Logistic Regression assumes a linear decision boundary; non-linear models (XGBoost, Neural Networks) may capture complex patterns better.

The dataset is static; a real production system requires continuous retraining on fresh incoming data.

No deployment infrastructure was set up as part of this project.

9.2 Future Work

1. Implement Random Forest and XGBoost for model comparison
2. Apply SHAP values for deeper feature-level explainability
3. Deploy the model as a Flask / FastAPI REST endpoint

4. Implement *k*-fold cross-validation for more robust evaluation
5. Explore SMOTE as an alternative class imbalance handling strategy

This internship with LaunchED Global provided the ideal environment to apply theoretical Machine Learning knowledge to a real-world business problem. Building a model from first principles rather than simply calling library functions delivered a much deeper understanding of how gradient descent, loss functions, and regularisation actually work under the hood.

References

1. IBM. (2019). Telco Customer Churn Dataset. IBM GitHub Repository. https://github.com/IBM/telco-customer-churn-on-icp4d

2. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

3. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.

4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

5. Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A pro t driven data mining approach. European Journal of Operational Research, 218(1), 211 229.

6. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825 2830.

7. Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357 362.

A Submission Checklist

Table A.1: LaunchED Capstone Submission Checklist

#	Task	Notes
1	Upload project folder to Google Drive	Code + report + resources
2	Set sharing to Anyone with link can view	Check before submitting
3	Record 5 10 min demo video (MP4)	Intro in English mandatory
4	Upload video to Google Drive	Separate from project folder
5	Post video on LinkedIn	Public post
6	Tag LaunchED Global on LinkedIn post	linkedin.com/company/launchedglobal
7	Tag your HOD on LinkedIn post	Share achievement
8	Write Google Review with project details	g.page/r/CeJX6LExaYbwEAE/review
9	Screenshot your Google Review	After submission
10	Upload screenshot to Google Form	From submission email
11	Fill and submit Final Google Form	All links required
12	Submit before 30 June 2026	No extensions

Support Email: support@launched.org.in

Phone: 08062178600

LinkedIn: https://in.linkedin.com/company/launchedglobal

B Software Environment

Table B.1: Required Software and Versions

Software	Minimum Version	Purpose
Python	3.9	Core language
NumPy	1.24	Model implementation
Pandas	1.5	Data manipulation
Matplotlib	3.6	Visualisation
Seaborn	0.12	Statistical plots
Scikit-learn	1.2	Benchmark + preprocessing
Jupyter Notebook	6.5	Development environment

```
pip install numpy pandas matplotlib seaborn scikit-learn
    jupyter
```

Listing B.1: Installation command